

Requirements: Observation-Driven Behavioral Graph Discovery for LovelaceSharp

Status: Requirements (lift from *Migration as Knowledge Compilation* white paper v0.3). This is the **requirements base** for the agent's work on LovelaceSharp. It defines *what* the system must do; it does **not** prescribe class names, file layout, or exact JSON fields.

Source of truth: the white paper, read with the correct emphasis — the system is understood by **executing it**, not by reading its source or its proofs.

Implementation substrate (decided): the tooling is written in **C#** in the LovelaceSharp solution and exposed as a **CLI with JSON-over-stdio** (the `Lovelace.Run` pattern), consumed by the DeepSeek Harness through a **thin bridge** — no business logic in JavaScript.

1. The idea, stated correctly

LovelaceSharp's engine is a **behavioral system**. We model it the way the white paper §6–§8 describes: treat the engine as an observation map over an input space, sample that space with **Monte Carlo**, execute each sample against the **real interface** (`Lovelace.Run`), canonicalize the outputs, cluster them into **behavior planes**, detect the **boundaries** between planes, and grow a **graph model** until it **converges with the actual system**.

The process is **purely computational**: samples are drawn, executed, reduced, and merged by code. Convergence is *measured against the actual system*, not declared by hand. The agent configures the domain and reads the resulting graph — it does **not** hand-build nodes or force convergence.

2. Non-negotiable principles

- **P1 — Observation-driven.** The graph is built exclusively from recorded executions. No node, edge, guard, or boundary enters the graph except as a consequence of observations.
- **P2 — Purely computational.** The loop *sample* → *execute* → *canonicalize* → *reduce* → *measure convergence* → *sample again* runs autonomously, with no per-step human/agent edits.
- **P3 — No artificial convergence.** The agent must not hand-seed laws, manually wire edges, or promote a hypothesis to a fact without observation. The only permitted agent inputs are the **sampling domain**, the **proposal distribution**, and the **convergence thresholds** — all configuration, none of it graph content.
- **P4 — Proofs are not the driver.** `Lovelace.Proofs/` (Lean) is out of scope for this graph. Behavior — observed output — is the oracle, not the theorems.
- **P5 — Determinism / reproducibility.** Seeded sampling plus deterministic canonicalization and reduction means the same domain + seed reproduces the same graph.
- **P6 — Logic lives in C#.** Sampling, canonicalization, reduction, boundary estimation, convergence, graph storage, and traversal are implemented in the C# solution — never in the harness bridge.

3. System under study and the oracle

- **S** is the LovelaceSharp engine (`Lovelace.Suite`), exercised through the non-interactive runner `Lovelace.Run` — the **real interface**.
 - One observation = run one Lovelace script, read the JSON envelope (`{ok, result{kind,typed}, variables[], diagnostics[], ...}`). Each call is a fresh engine: there is no hidden cross-run state except what the script itself establishes (e.g. `_` = last result).
-

4. Implementation substrate (requirement)

- **R-IMPL-1 — Logic in C#**. The tooling (sampling, canonicalization, reduction, boundary estimation, convergence, graph storage, traversal) is implemented in **C#**, within the LovelaceSharp solution — not in the harness.
 - **R-IMPL-2 — JSON API**. It is exposed as a **JSON API**, a CLI with JSON-over-stdio following the `Lovelace.Run` pattern, so consumers drive it by sending JSON and reading JSON.
 - **R-IMPL-3 — Thin bridge only**. The DeepSeek Harness consumes the API through a thin bridge (the same pattern as the existing `Lovelace` tool) that performs transport only: JSON marshaling and process invocation. It contains no sampling, reduction, or graph logic.
 - **R-IMPL-4 — Capability coverage**. The API must expose the capabilities required in §5–§10 — sampling, reduction, convergence, and query — and persist the graph. Concrete command names and the storage encoding are **design decisions**, out of scope here.
-

5. Input domain Ω and Monte Carlo sampling (paper §7.4)

- Ω is the space of Lovelace scripts, parameterized by a generator. Minimum concrete shape:
 - **operands** drawn from numeric domains — naturals, integers, reals, periodic fractions, large magnitudes, ranges, and (later) vectors/arrays;
 - **operations** — `+ - * / ^ %`, comparisons, and built-ins (`sqrt`, `pi`, `divrem`, `sum`, ...);
 - **statement forms** — single expression, assignment, and multi-statement sequences.
 - Sampling draws $z_i \sim q(z)$ from an **explicit proposal distribution** and records, for each sample, the coordinate z_i , the canonical output σ_i , the density $q(z_i)$, and the **seed**. q is kept so samples can be reweighted later — a convenience sample is never presented as real frequency.
 - The sampler uses a **seeded RNG**; the seed is part of the reproducible state.
-

6. Canonical observation σ (paper §4.1)

σ is a deterministic reduction of the envelope that keeps the **behavior class** and discards noise:

- success → the result type + typed value (e.g. `Real + 0.(3) (Real)`);
- failure → the error class (message + diagnostic line/column), not transient formatting;
- **excluded**: `revision`, `elapsed`, plot SVG bytes, variable iteration order.

Two executions are the same graph state iff their σ is equal under the current observation model (purpose-relative equivalence, §4.2).

7. Graph model (paper §3, §7.7, §8.3)

- **Node = behavior plane.** An equivalence class of observations sharing one canonical output, clustered and checked for local connectedness in Ω . One node represents the supported *region*, not one sample point (§8.3). Each node carries its σ , sample support, and confidence.
- **Edge = boundary / adjacency.** Two planes are adjacent when a local perturbation crosses between them. A fitted **guard** (e.g. `a < 0`, `divisor == 0`, `magnitude ≥ threshold`) is recorded with the samples that bound it (§3.2, §7.2).
- **Evidence = the samples.** Every sample's coordinate, σ , seed, and weight stay attached as provenance; sample clouds are provenance, not one node each (§7.7).
- **Frontier = explicitly unresolved.** Unsourced regions, low-support planes, and boundaries that lack counterexamples on both sides (§5 open-world rule).

Confidence levels follow §5.1: `hypothesized < observed < repeated < bounded < conformant < proven`, and a fact is only promoted by more observation — never by an edit.

8. Convergence — measured, computational (paper §8.2)

Convergence is a property of the graph against the actual system, computed from observations:

- **C1 — Plane saturation.** Distinct-plane count as a function of samples reaches a plateau; the new-plane discovery rate over the last `k` batches $\rightarrow 0$.
- **C2 — Boundary stability.** A fitted guard stops changing across repeated refinement and across different perturbation step sizes `h`.
- **C3 — Prediction agreement.** On held-out samples the model's assigned plane equals the actual σ . (Trivially exact inside a plane by construction; the meaningful check is **near boundaries** and in previously unsampled neighborhoods.)
- **C4 — Coverage.** Weighted sample mass per plane and per frontier; weakly-sampled dimensions are reported, not hidden.

Stopping criterion. The loop stops when C1–C4 reach configured thresholds ("decent convergence"). The thresholds are explicit, recorded inputs — the agent sets them once, then the loop runs to completion on its own.

9. The loop (computational pipeline)

```
config( $\Omega$ , q, thresholds, seed)
  → sample batch  $\{z_i \sim q\}$ 
  → execute each  $z_i$  against Lovelace.Run
  → canonicalize to  $\{\sigma_i\}$ 
  → reduce: cluster into planes, detect boundaries, fit guards, mark frontiers
  → merge into the graph (idempotent, by stable identity)
  → measure C1–C4
  → if not converged: bias next batch toward frontiers/high-variance neighborhoods, repeat
```

This loop runs autonomously to the configured threshold; its individual steps (sample, reduce, query) are also individually invocable through the API for inspection and control. Every step is a deterministic function of the

sample set (P5). The agent only sets `config` and reads the graph + metrics; it does not edit the graph between iterations (P2, P3).

10. Traversal / understanding (paper §10, §11)

- Query planes, boundaries, and frontiers; walk from a plane to its boundary neighbors, their guards, and their supporting evidence.
- The graph answers "*what does the engine do in this region, and where does that behavior change?*" — the behavioral structure of the code, **discovered**, not asserted.

11. Harness integration (paper §2, §9) — thin bridge only

- The DeepSeek Harness consumes the C# CLI through a **thin bridge**, mirroring the existing `loveLace` tool: the bridge registers DSH tools that marshal JSON and spawn the CLI via DSH `subprocess`. No sampling/reduction/graph logic lives in the bridge (P6).
- DSH supplies process execution (`subprocess`), filesystem access (`fs`) for the persisted graph and sample log, and session/event provenance on evidence.
- The long-running `converge` command may be run as a **DSH background job**; the graph file is the durable product.
- The bridge's tool registrations belong to the plugin fiber (reversible on stop/update). No second harness.

12. Acceptance criteria

1. Given a domain config + seed, the CLI produces a graph whose planes are exactly the distinct canonical outputs observed — **no hand-added nodes** (P1).
2. Re-running the CLI with the same seed + config reproduces the same graph (P5, across the process boundary).
3. The new-plane discovery rate falls below the configured threshold within the configured sample budget (C1 — convergence).
4. At least one genuine behavior boundary is localized to a guard with counterexamples on both sides — e.g. natural-subtraction underflow, division-by-zero, or the integer→periodic-real transition (C2, §7).
5. `query` reports each plane's boundary neighbors, guards, and evidence, and lists remaining frontiers (C4, §10).
6. The harness bridge contains **no** sampling/reduction/graph logic — transport only (R-IMPL-3).

13. Traceability

White-paper section	Lifted to
§4 Witnessed state / canonical observation	§6
§5 Evidence, provenance, confidence, open-world	§7, §8
§6 Grey-box E2E exploration	§3, §5
§7 Monte Carlo + boundary search (finite differences)	§5, §7, §8, §9
§8 State inference, planes, semantic coverage	§7, §8

White-paper section	Lifted to
\$9 DSH as execution substrate	\$11
\$10/\$11 Graph as IR + interpretation	\$9, \$10

14. Out of scope (explicit)

- **Lean proofs** as an anchor or backbone (per direction: proofs are not the driver of this graph).
 - **Static source analysis** to seed the graph — optional later phase, never the primary evidence.
 - **Backends** (code generation, formal-model lowering, conformance tests) — later phases, only after the graph has converged against the actual system.
 - **Any tooling logic in JavaScript** — the C# CLI is the sole implementation of the behavior (P6); the harness bridge is transport only.
-

15. Open questions to confirm before implementation

1. **The concrete Ω for the first convergence run** — e.g. pairwise `a op b` over naturals + integers
 - reals (with negatives and fractions) to expose widening/underflow/periodic boundaries, then widening to built-ins and arrays?
2. **The C1–C4 thresholds** — what counts as "decent convergence" (e.g. new-plane rate < 1% over the last 1000 samples, and boundary guards stable across two refinement passes)?
3. **σ granularity** — result-only, or also the named-variable set / kind for multi-statement scripts?
4. **Sample budget / cost** — each sample is a process spawn; what budget per run is acceptable, and should the convergence loop run as a background job?